Customer Shopping Behavior Analysis

Python, PostgreSQL and Power BI project report. Prepared by Devendra Choudhary.

Summary

I cleaned a 3,900-row retail customer dataset in Python, answered ten business questions in PostgreSQL, and built a Power BI dashboard on top of it. The headline finding is that **spending is almost identical across every segment I tested**. Average purchase is about $60 whether the customer is male or female, young or senior, subscribed or not, discounted or not. Revenue differences between segments come from **how many customers** each segment has, not from how much each one spends.

Metric	Value
Customers (orders)	3,900
Total revenue	$233,081
Average purchase amount	$59.76 (median $60)
Average review rating	3.75 / 5
Subscribers	1,053 (27.0%)
Orders with a discount	1,677 (43.0%)
Customers with more than 5 previous purchases	3,476 (89.1%)

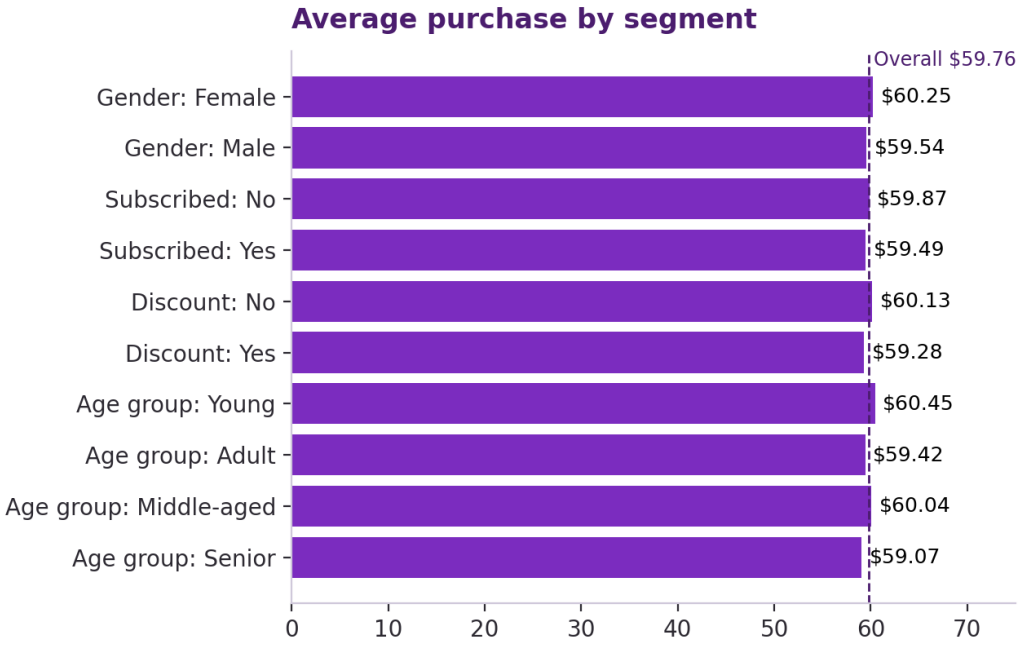


Figure 1. Average purchase per order for every segment tested. All sit within about $1.40 of the $59.76 overall average.

1. Data and cleaning

The raw file has 3,900 rows and 18 columns, with one customer per row and no duplicate rows. Cleaning was done in Python (pandas) in Google Colab:

- **Missing values:** only *Review Rating* had nulls (37 rows, 0.9%). They were filled with the median rating of the same category, so a missing rating for a Footwear item is not filled from Clothing.
- **Column names:** converted to lowercase with underscores, and *purchase_amount_(usd)* renamed to *purchase_amount*, so SQL needs no quoting.
- **age_group (new):** the age column was split into four equal-sized quartile bands using pd.qcut: Young (18-31), Adult (32-44), Middle-aged (45-57) and Senior (58-70).
- **purchase_frequency_days (new):** the text frequency (Weekly, Monthly, Quarterly, ...) was mapped to a number of days, so frequency can be compared numerically.
- **Dropped promo_code_used:** I checked all 3,900 rows and it is identical to *discount_applied*, so one column was redundant.
- **Load:** the cleaned file (19 columns) was exported to CSV and loaded into a local PostgreSQL database with pandas and SQLAlchemy. The row count after loading (3,900) matched the DataFrame.

Python (cleaning, run in Google Colab)

```
import pandas as pd

df = pd.read_csv('customer_shopping_behavior.csv')
df.info(); df.describe(include='all'); df.isnull().sum()

# 1. Fill missing ratings with the median of the same category
df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(
    lambda x: x.fillna(x.median()))

# 2. Clean column names
df.columns = df.columns.str.lower().str.replace(' ', '_')
df = df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})

# 3. Age groups (four equal-sized quartile bands)
labels = ['Young', 'Adult', 'Middle-aged', 'Senior']
df['Age_group'] = pd.qcut(df['age'], q=4, labels=labels)

# 4. Purchase frequency in days
frequency_mapping = {'Fortnightly': 14, 'Monthly': 30, 'Weekly': 7, 'Quarterly': 90,
                     'Bi-Weekly': 14, 'Annually': 365, 'Every 3 Months': 90}
df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)

# 5. promo_code_used is identical to discount_applied, so drop it
print((df['discount_applied'] == df['promo_code_used']).all())   # True
df = df.drop('promo_code_used', axis=1)

df.to_csv('customer.csv', index=False)
```

Python (load into PostgreSQL, run locally)

```
import os, pandas as pd
from sqlalchemy import create_engine, text
from sqlalchemy.engine import URL

url = URL.create("postgresql+psycopg2", username="postgres",
                 password=os.environ["PG_PASSWORD"],   # keep passwords out of the code
                 host="localhost", port=5432, database="customer_behaviour")
engine = create_engine(url)

df = pd.read_csv("customer.csv")
df.to_sql("customer", engine, if_exists="replace", index=False,
          chunksize=1000, method="multi")

with engine.connect() as conn:
    print(conn.execute(text("SELECT COUNT(*) FROM customer")).scalar())   # 3900
```

2. Business questions answered in SQL

All queries run on the table *customer* (the loaded table, renamed with ALTER TABLE customer_shopping_behavior RENAME TO customer).

Q1. Revenue by gender

SQL

```
SELECT gender, SUM(purchase_amount) AS revenue_generated
FROM customer
GROUP BY gender;
```

Gender	Customers	Revenue	Average purchase
Male	2,652	$157,890	$59.54
Female	1,248	$75,191	$60.25

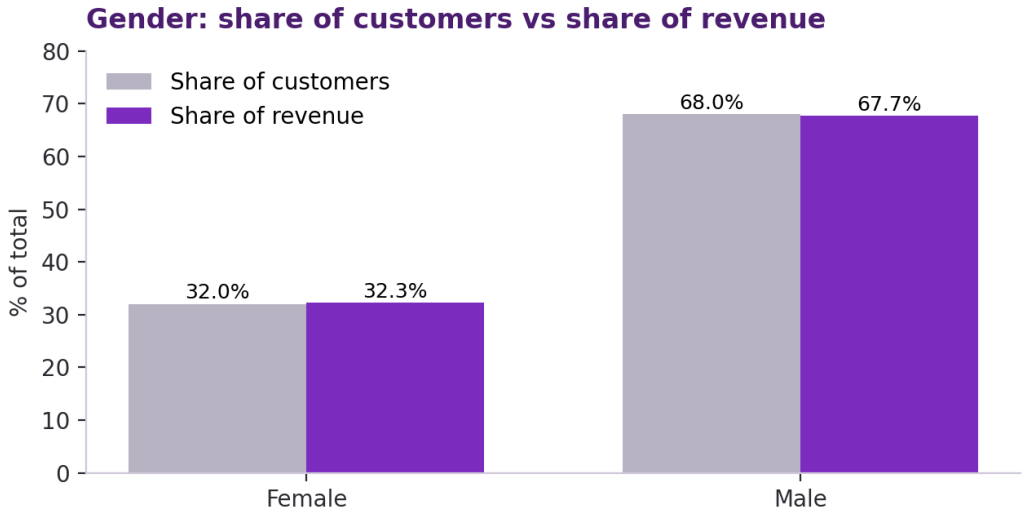


Figure 2. Each gender's share of revenue matches its share of customers almost exactly.

Males bring 67.7% of revenue because they are 68% of customers. Per customer, females spend slightly more ($0.71), which is small enough to be noise.

Q2. Customers who used a discount but still spent above the average

SQL

```
SELECT customer_id, purchase_amount
FROM customer
WHERE discount_applied = 'Yes'
  AND purchase_amount >= (SELECT AVG(purchase_amount) FROM customer);
```

839 of the 1,677 discounted orders (50.0%) were at or above the $59.76 average. A discount does not push people into smaller baskets: discounted orders average $59.28 versus $60.13 without a discount.

Q3. Top 5 products by average review rating

SQL

```
SELECT item_purchased, ROUND(AVG(review_rating::numeric), 2) AS average_review_rating
FROM customer
GROUP BY item_purchased
ORDER BY AVG(review_rating::numeric) DESC     -- order by the unrounded average
LIMIT 5;
```

Product	Avg rating
Gloves	3.86
Sandals	3.84
Boots	3.82
Hat	3.80
Skirt	3.78

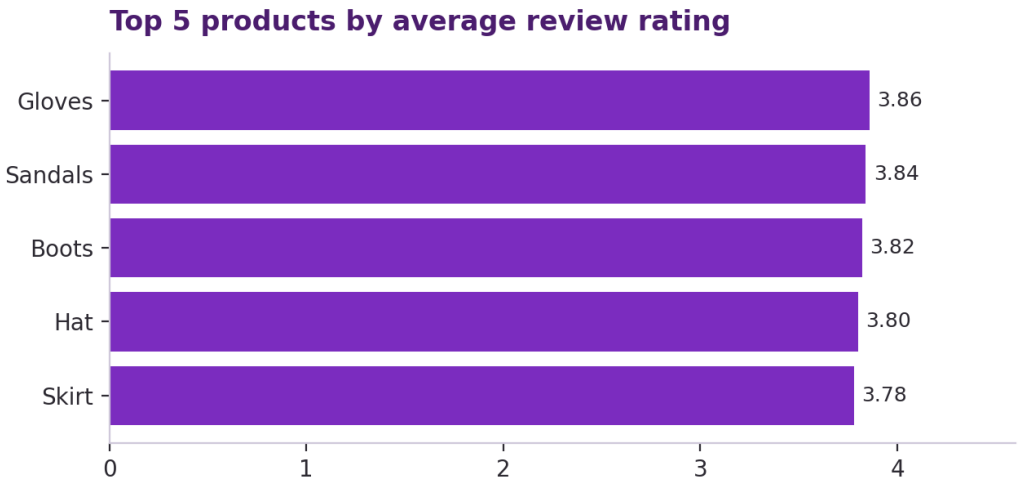


Figure 3. The top five are separated by only 0.08 rating points.

The lowest are Shirt (3.62), Jeans (3.65) and Blouse (3.68), so the weakest-rated products are all core Clothing items. The whole range is narrow (about 3.6 to 3.9). Ordering by the rounded value can swap tied products at the cut-off: Skirt (3.785), T-shirt (3.778) and Handbag (3.775) all show 3.78, so order by the unrounded average as above.

Q4. Standard vs Express shipping

SQL

```
SELECT shipping_type, ROUND(AVG(purchase_amount), 2) AS avg_purchase_amount
FROM customer
WHERE shipping_type IN ('Express', 'Standard')
GROUP BY shipping_type;
```

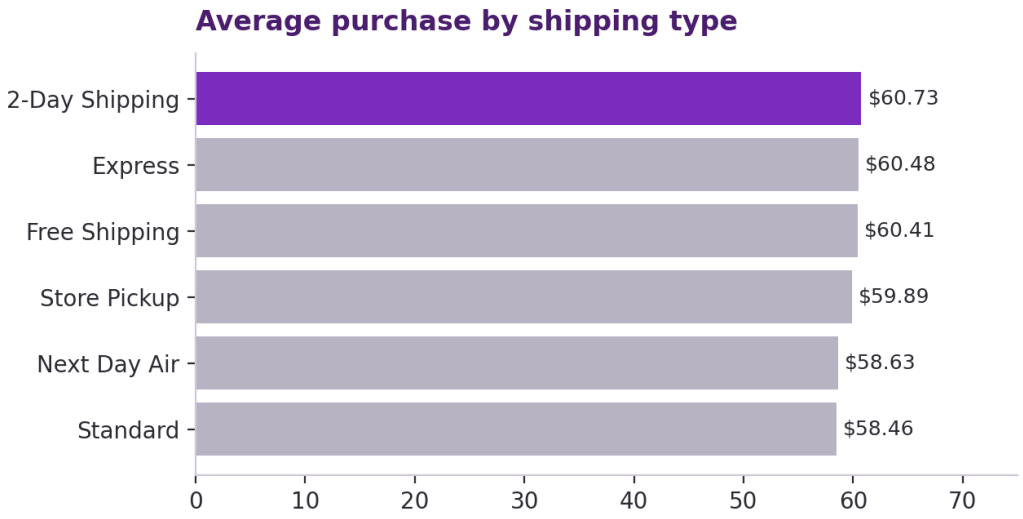


Figure 4. Average purchase for all six shipping types (the query above compares just Express and Standard).

Express averages $60.48 and Standard $58.46, a gap of $2.02 (3.5%). Across all six shipping types the average sits between $58.46 and $60.73, so shipping type has only a small link to spend.

Q5. Do subscribers spend more?

SQL

```
SELECT subscription_status,
       COUNT(customer_id)            AS total_customer,
       ROUND(AVG(purchase_amount),2) AS avg_revenue,
       ROUND(SUM(purchase_amount),2) AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue, avg_revenue DESC;
```

Status	Customers	Average spend	Total revenue
No	2,847	$59.87	$170,436
Yes	1,053	$59.49	$62,645

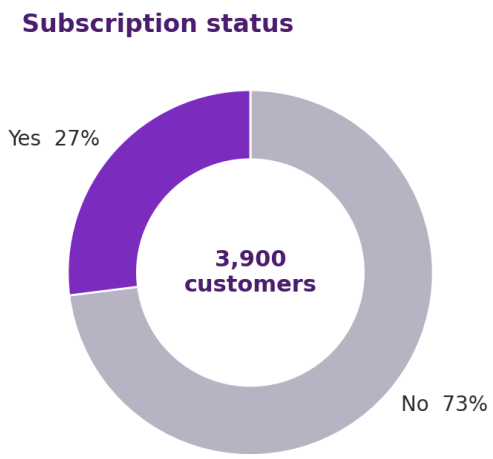


Figure 5. 27% of customers are subscribers.

No. Subscribers spend about the same per order ($0.38 less). Their ratings are identical (3.75 both). Subscribing does not currently translate into higher spend in this data.

Q6. Top 5 products by share of purchases made with a discount

SQL

```
SELECT item_purchased,
       ROUND(100 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END)
             / COUNT(*), 2) AS discount_rate
FROM customer
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

Product	Discount rate
Hat	50.00%
Sneakers	49.66%
Coat	49.07%
Sweater	48.17%
Pants	47.37%

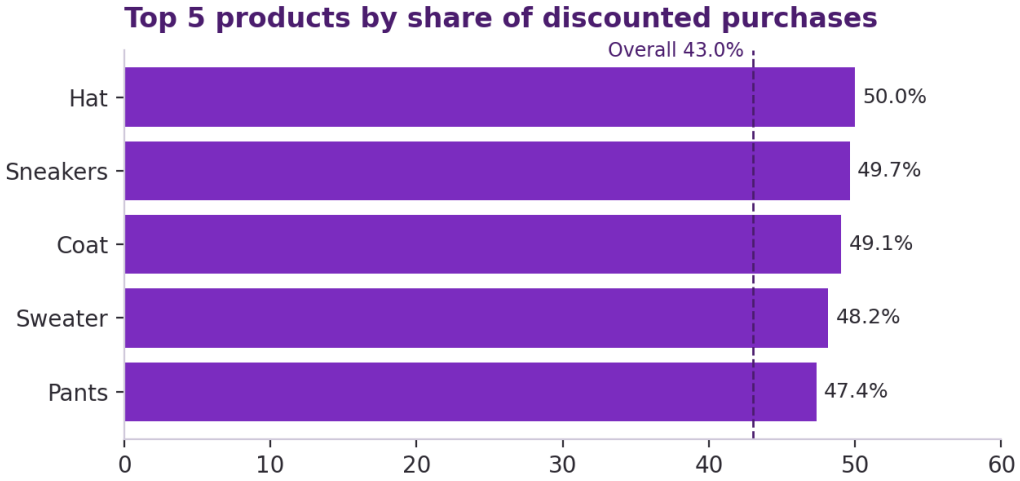


Figure 6. The five most-discounted products against the 43.0% overall rate.

The overall discount rate is 43.0%, so these five run about 4 to 7 points above average.

Q7. Customer segments by previous purchases

SQL

```
WITH customer_type AS (
  SELECT customer_id, previous_purchases,
         CASE WHEN previous_purchases = 1 THEN 'New'
              WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'
              ELSE 'Loyal' END AS customer_segment
  FROM customer
)
SELECT customer_segment, COUNT(*) AS "Number of Customers"
FROM customer_type
GROUP BY customer_segment;
```

Segment	Rule	Customers
New	1 previous purchase	83 (2.1%)
Returning	2-10	701 (18.0%)
Loyal	more than 10	3,116 (79.9%)

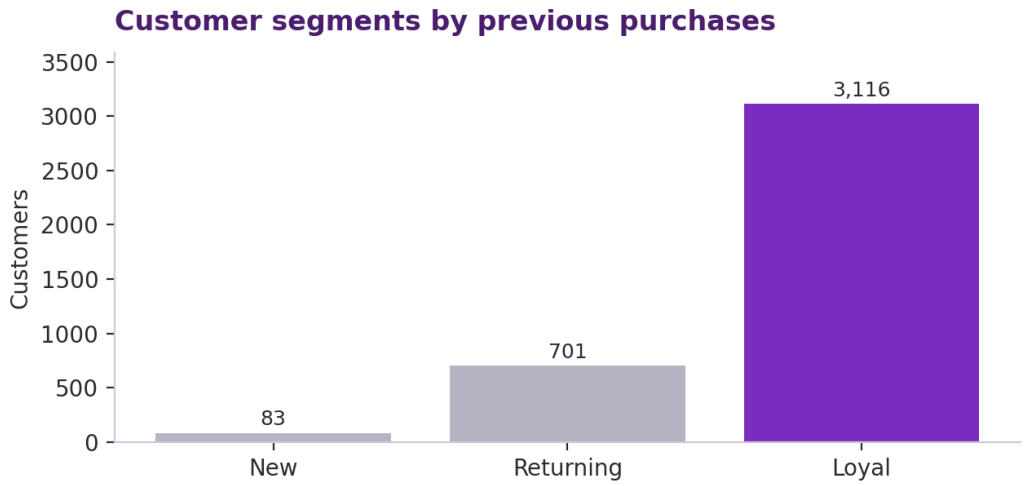


Figure 7. With these cut-offs, 80% of customers fall in one segment.

This works, but the thresholds make 80% of customers "Loyal", so the label tells us little. Previous purchases range from 1 to 50 and are spread fairly evenly, so a better split would use quartiles or round cut-offs such as 1-10, 11-25, 26-40 and 41-50.

Q8. Top 3 most purchased products per category

SQL

```
WITH item_counts AS (
  SELECT category, item_purchased,
         COUNT(customer_id) AS total_orders,
         ROW_NUMBER() OVER (PARTITION BY category
                            ORDER BY COUNT(customer_id) DESC) AS item_rank
  FROM customer
  GROUP BY category, item_purchased
)
SELECT item_rank, category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <= 3;
```

Category	#1	#2	#3
Accessories	Jewelry (171)	Belt (161)	Sunglasses (161)
Clothing	Blouse (171)	Pants (171)	Shirt (169)
Footwear	Sandals (160)	Shoes (150)	Sneakers (145)
Outerwear	Jacket (163)	Coat (161)	(only two products)

Outerwear has only two products, so it shows two rows. Counts within a category are close, so no single product dominates. Where counts tie (Jewelry/Blouse/Pants, Belt/Sunglasses), ROW_NUMBER picks an arbitrary order; use RANK or DENSE_RANK if ties should show together.

Q9. Are repeat buyers (more than 5 previous purchases) more likely to subscribe?

The original query returns counts only:

SQL

```
SELECT subscription_status, COUNT(customer_id) AS repeat_buyers
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;      -- No 2,518 / Yes 958
```

The raw counts can mislead because non-subscribers are 73% of everyone. The rate is the right comparison:

SQL

```
SELECT CASE WHEN previous_purchases > 5 THEN 'Repeat' ELSE 'Not repeat' END AS buyer_type,
       COUNT(*) AS customers,
       ROUND(100.0 * SUM(CASE WHEN subscription_status = 'Yes' THEN 1 ELSE 0 END)
             / COUNT(*), 1) AS subscription_rate_pct
FROM customer
GROUP BY 1;
```

Group	Customers	Subscription rate
Repeat buyers (more than 5)	3,476	27.6%
5 or fewer previous purchases	424	22.4%

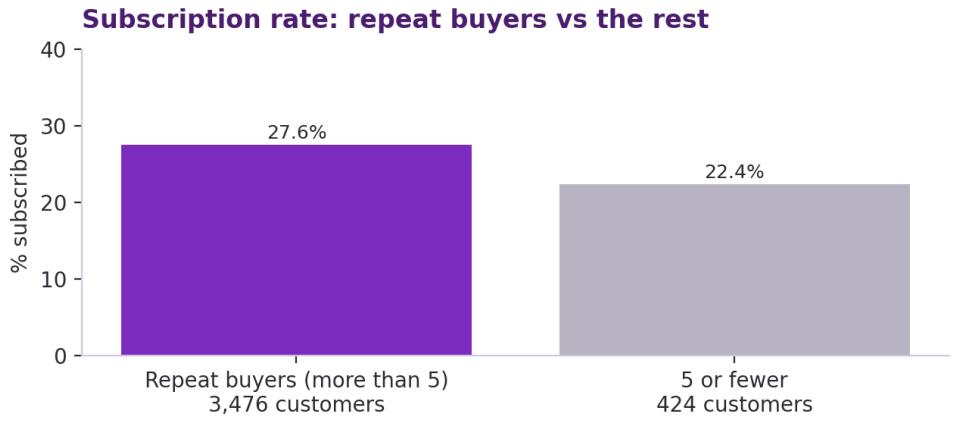


Figure 8. Repeat buyers subscribe more often, but the comparison group is small.

Repeat buyers subscribe about 5 points more often, but the comparison group is small (424 customers), so treat it as a hint rather than a proven effect.

Q10. Revenue by age group

SQL

```
SELECT "Age_group", SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY "Age_group"
ORDER BY total_revenue DESC;
```

Age group	Customers	Revenue	Share	Average purchase
Young (18-31)	1,028	$62,143	26.7%	$60.45
Middle-aged (45-57)	986	$59,197	25.4%	$60.04
Adult (32-44)	942	$55,978	24.0%	$59.42
Senior (58-70)	944	$55,763	23.9%	$59.07

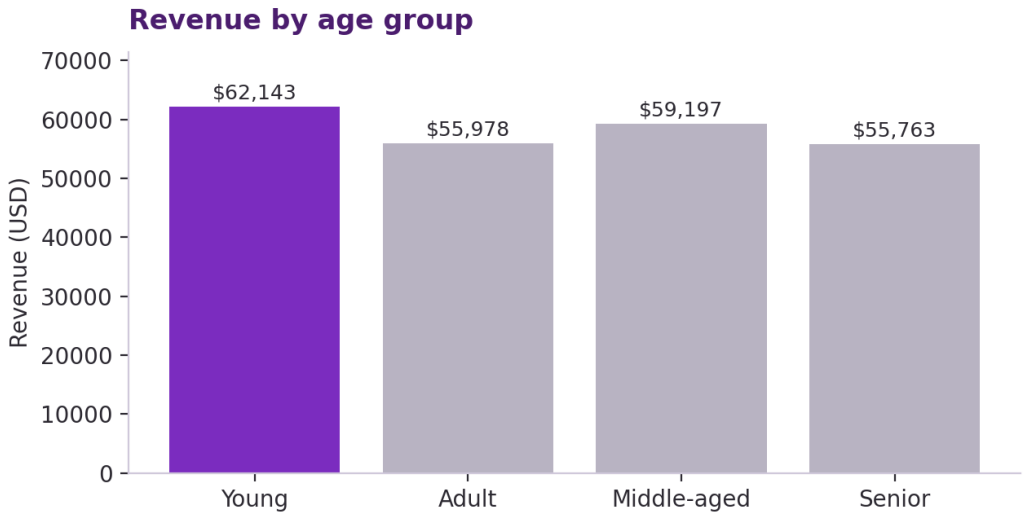


Figure 9. Revenue by age group, in age order.

Young leads only because it has the most customers. Average spend per customer is almost flat across the four groups.

3. Extra breakdowns

By category

Category	Customers	Revenue	Share	Avg rating
Clothing	1,737	$104,264	44.7%	3.72
Accessories	1,240	$74,200	31.8%	3.77
Footwear	599	$36,093	15.5%	3.79
Outerwear	324	$18,524	7.9%	3.75

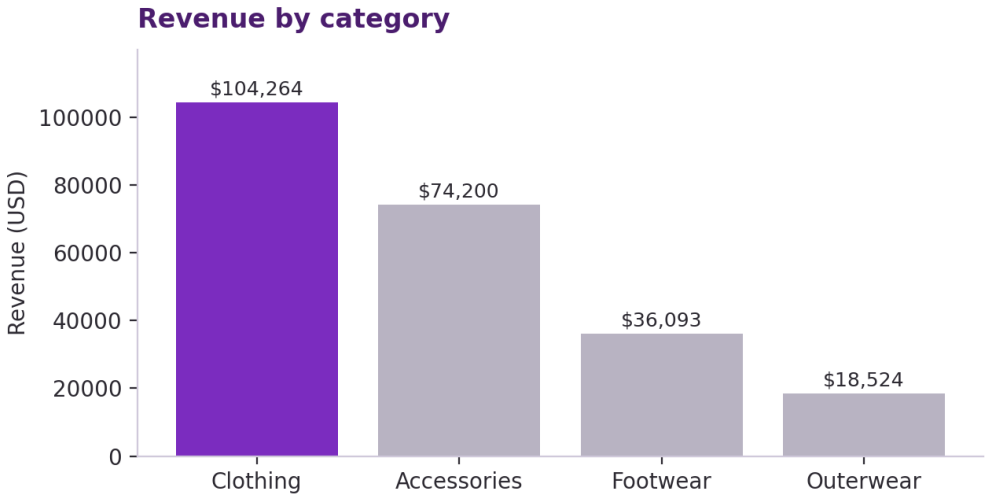


Figure 10. Revenue by category. Clothing is nearly half of the total.

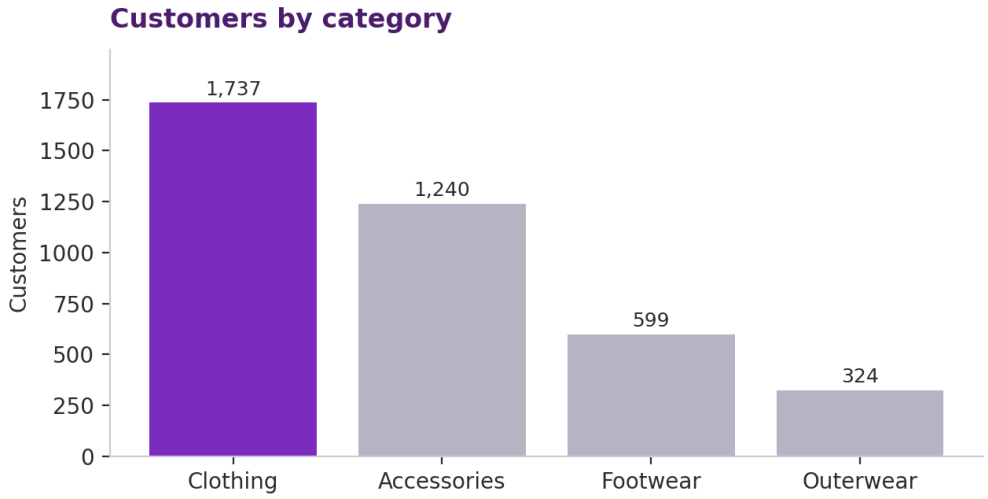


Figure 11. Customers by category. The shape matches revenue because spend per order is flat.

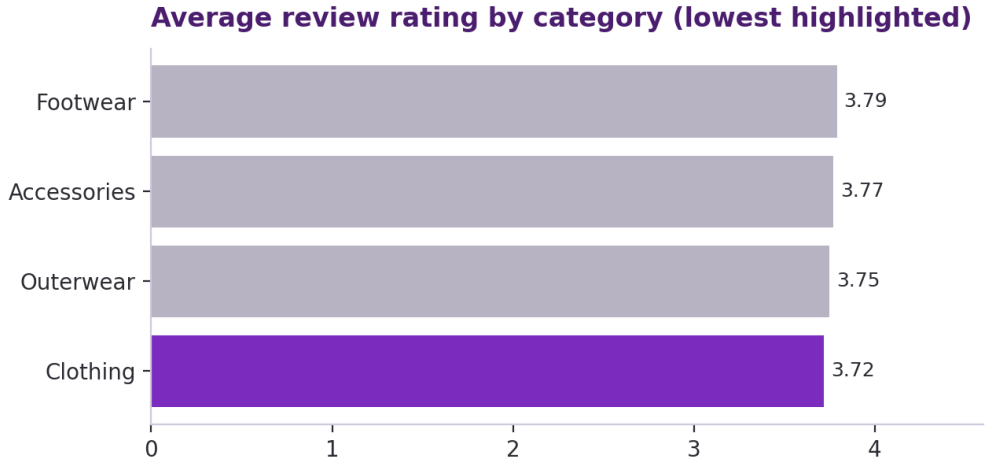


Figure 12. Average rating by category. Clothing has the lowest.

Clothing is nearly half of revenue and has the lowest rating. Outerwear has the lowest average purchase ($57.17).

SQL

```
SELECT category,
       COUNT(*)                          AS customers,
       SUM(purchase_amount)              AS revenue,
       ROUND(100.0 * SUM(purchase_amount) / SUM(SUM(purchase_amount)) OVER (), 1) AS share_pct,
       ROUND(AVG(review_rating::numeric), 2) AS avg_rating
FROM customer
GROUP BY category
ORDER BY revenue DESC;
```

By season: Fall is the strongest ($60,018, 25.7%, average $61.56) and Summer the weakest ($55,777, 23.9%, average $58.41). The spread is small.

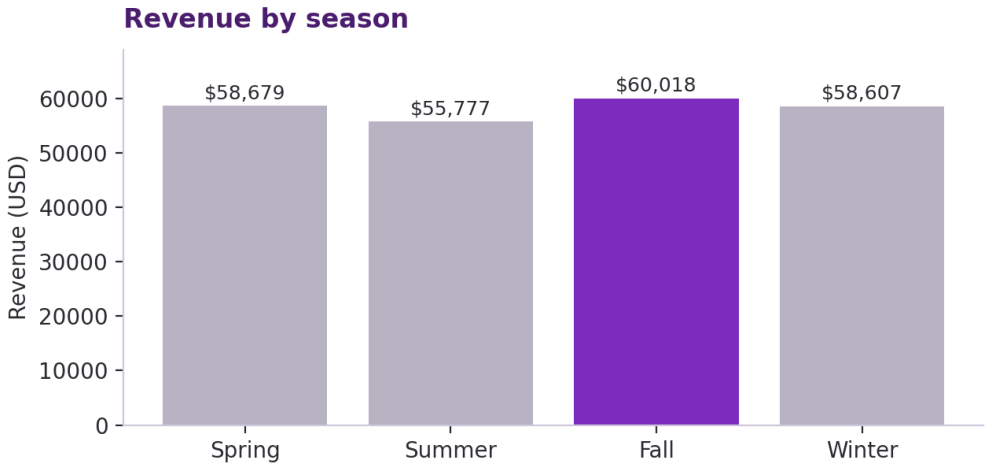


Figure 13. Revenue by season.

By location: across 50 states, the top five by revenue are Montana ($5,784), Illinois, California, Idaho and Nevada. The lowest are Kansas ($3,437), Hawaii and Florida. The gap is mostly customer count (63 to 96 per state).

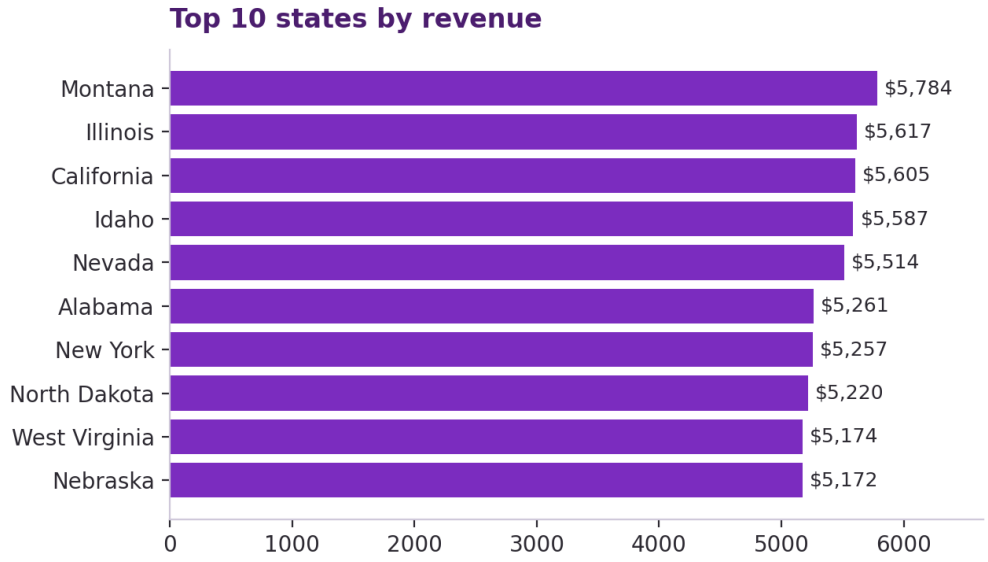


Figure 14. Top 10 states by revenue.

SQL

```
SELECT location, COUNT(*) AS customers, SUM(purchase_amount) AS revenue
FROM customer
GROUP BY location
ORDER BY revenue DESC
LIMIT 10;
```

Correlations with purchase amount: age (-0.01), previous purchases (0.01) and review rating (0.03) are all essentially zero. Nothing in this dataset predicts how much a customer spends.

Python

```
print(df[['purchase_amount', 'previous_purchases', 'review_rating', 'age']].corr()['purchase_amount'])
```

4. What I would do with these findings

1. **Don't expect subscription to lift spend yet.** Subscribers spend the same as non-subscribers, so the program needs a real benefit (members-only bundles, free shipping thresholds) before it can be called a revenue driver.
2. **Grow customer count, not basket size.** Spend per order is flat at about $60 everywhere, so reaching more customers, especially in Footwear and Outerwear, is where revenue growth would come from. Outerwear is only 8% of revenue.
3. **Look at Clothing quality.** It is 45% of revenue but has the lowest category rating, and Shirt, Jeans and Blouse are the three lowest-rated products. Fix these first.
4. **Review discount strategy.** 43% of orders are discounted, yet discounted orders are not larger. Test whether the discounts are needed.
5. **Rebuild the loyalty segments** with quartile-based cut-offs so "Loyal" means something.

5. Power BI dashboard

The dashboard has KPI cards (customers, average purchase, average rating), a subscription donut, revenue by category, revenue by age group, and slicers for subscription status, gender, category and shipping type. I checked its numbers against the database and found these issues to fix:

- **Revenue By Category (right-hand copy):** it shows about 1,737 for Clothing, which is the customer count, not revenue. The correct revenue figure is $104,264. Retitle it "Customers by Category" or delete it.
- **Sales By Age Group:** the bars sit near 2.0M, which is impossible when total revenue is $233K. The value matches the sum of customer_id (for example Young = 2,005,142). Replace it with SUM(purchase_amount) or a customer count.
- **Typo:** "Revvenue By Age group" should read "Revenue by Age Group".
- **Sort order:** sort age groups Young, Adult, Middle-aged, Senior instead of by value.
- **Slicer style:** make all four slicers the same style, and keep the whole dashboard on one color palette.

Measures that give the correct numbers (table name assumed to be *customer*):

DAX

```
Total Revenue     = SUM(customer[purchase_amount])
Customers         = DISTINCTCOUNT(customer[customer_id])
Avg Purchase      = AVERAGE(customer[purchase_amount])
Avg Rating        = AVERAGE(customer[review_rating])

Subscription Rate =
DIVIDE(
    CALCULATE(DISTINCTCOUNT(customer[customer_id]),
              customer[subscription_status] = "Yes"),
    DISTINCTCOUNT(customer[customer_id])
)
```

6. Limitations

- The data is one row per customer with no dates, so I could not analyse trends over time, year-over-year growth or true customer lifetime value.
- Nearly every segment difference is small, and I did not run significance tests. Treat the differences as descriptive, not proven.
- Missing ratings were filled with category medians, which slightly smooths the rating data.
- Age groups are quartile bands, so the group names (Young, Adult, Middle-aged, Senior) are labels of convenience, not standard demographic definitions.

7. Suggested slide outline

Each figure is also supplied as a separate PNG (see the charts file) so you can drop it straight onto a slide.

Slide	Title	Use
1	Project overview	Summary text, tools
2	Key numbers	KPI table (customers, revenue, average purchase, rating)
3	Data cleaning	Cleaning bullets, Python cleaning code (trimmed)
4	Spending is flat	Figure 1 (the headline insight)
5	Revenue by category	Figures 10 and 11, Clothing is 45% of revenue
6	Do subscribers spend more?	Q5 table, Figure 5
7	Customer segments and loyalty	Figures 7 and 8, Q7 and Q9 code
8	Products	Figures 3 and 6 (top rated, most discounted)
9	Where and when	Figures 13 and 14 (season, states)
10	Dashboard	Power BI screenshot, DAX measures
11	Recommendations and limits	Section 4 and Section 6

Tools

Python (pandas, SQLAlchemy, psycopg2) for cleaning and loading, PostgreSQL and pgAdmin for analysis, Power BI for the dashboard, Google Colab for the notebook.